

i-Telex

Communication Specification

Content

1	Content	3
2	Communication between i-telex stations	4
2.1	Basic characteristics	5
2.2	Packet types.....	6
2.2.1	Heartbeat packet.....	6
2.2.2	Direct dial packet	6
2.2.3	Baudot data packet	7
2.2.4	Communication end packet.....	7
2.2.5	Communication reject packet.....	8
2.2.6	Acknowledge packet	8
2.2.7	Version packet.....	9
2.2.8	Self test packet	9
2.2.9	Remote configuration packet	9
2.3	Examples	10
3	Communication between i-telex station and subscriber server.....	12
3.1	Basic characteristics	12
3.2	Packet types.....	12
3.2.1	Client_update	12
3.2.2	Address_confirm	13
3.2.3	Peer_query.....	13
3.2.4	Peer_not_found	13
3.2.5	Peer_reply_v1	14
3.2.6	Sync_FullQuery	15
3.2.7	Sync_Login	15
3.2.8	Acknowledge.....	15
3.2.9	End_of_List.....	15
3.2.10	Peer_serach	16
3.2.11	Error message.....	16
3.2.12	Sequence Examples.....	16
3.3	Peer query in ASCII format	17
4	Backup: Basics of the communication.....	19

Pictures

Abbildung 1: Example structure	19
--------------------------------------	----

Tables

Table 1: Packet types and codes.....	6
Table 2: Coding of extension numbers	7
Table 3: Reject codes	8
Table 4: Complete example.....	10
Table 5: Simplified example	11

1 Content

This document describes the communication used at the IP port of the i-telex system.

There are basically two channels:

1. The communication between two i-telex users (station). This is subject of chapter 2.
2. The communication between the calling i-telex user (station) and the i-telex subscriber server, which is performed to get the IP address of the called station. This is subject of chapter 3.

2 Communication between i-telex stations

i-telex stations try to enable stations to simply interconnect two telexes thru the internet. The Protocol is TCP based and follows the same rules as the normal telex does.

It may be reflected by a state machine with the states “disconnected”, “connected unknown”, “connected proposed”, “connected” “texting ascii”, “texting baudot”, and “disconnected baudot”.

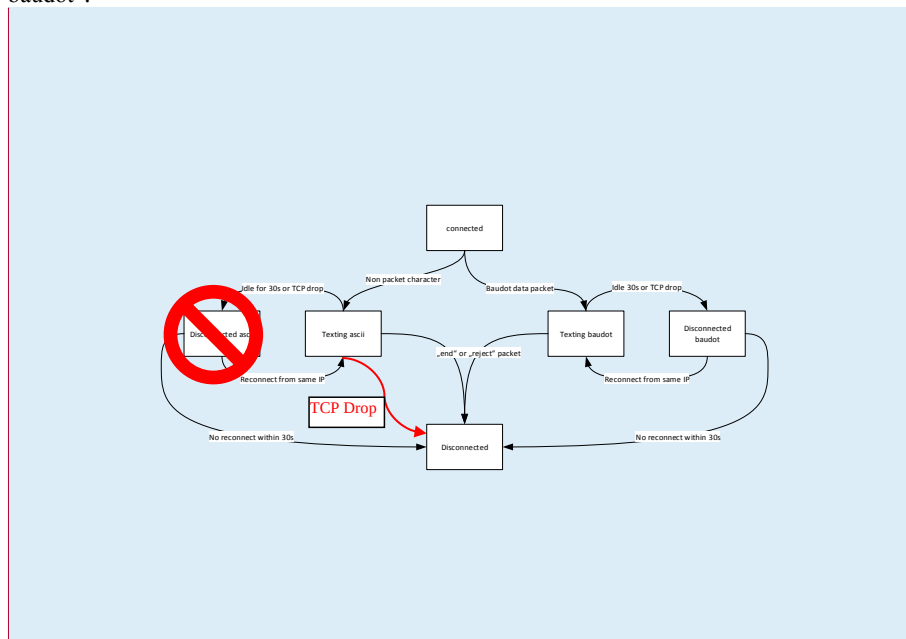


Figure 1: state diagram of an i-telex host

In the state disconnected the i-telex service waits for incoming connects on the designated TCP port.

As soon as there is an incoming TCP connect the state changes to connected. In the connected phase the following packets are allowed: “Version”, “Direct dial”, “remote config”, “selftest”, “reject”, “end” and plain ascii characters.

When in connected state the first packet not starting with a character in the range 0x00 to 0x09 and 0x10 to 0x1F triggers the “texting ascii” state. In this state all characters sent will be interpreted as pure ASCII transmission to be sent to the telex. ~~The state machine will remain in this state until an “end” or “rejected” packet is received or the TCP connection drops.~~

When in “connected” or “texting ascii” state and a “baudot data” packet is received the state machine changes to “texting baudot”. In this mode either a “baudot data”, “acknowledge”, or “heartbeat” packet must be sent at least every 15s. If no packet is received for more than 30s the TCP connection should be dropped and changed to the “disconnected baudot” state.

Kommentiert [SF1]: Den Modus gibt es nicht, da es im Ascii-Modus kein „Verbindungs-Ende-Zeichen“ gibt.

Kommentiert [GM2]: Pfeil fehlt disconnected->“connected unknown” resp. “connected proposed”

Kommentiert [Son3R2]: Und es fehlen die Pfeile von Texting Ascii zu Texting Baudot und zurück.

Kommentiert [Son4]: Nein, zu jeder beliebigen Zeit ist ein Wechsel erlaubt. Daher kommen die gängigen ASCII-Steuercodes wie CR und LF nicht als Befehlscode im „Baudot-Texting“ modus vor.

Okay, wenn es sein muss (ich frag mich immer noch warum), wird der wechsel verboten.

Ein Vorteil hätte die Zulässigkeit eines Wechsels: Ein Programm, das die eigentlichen Druckdaten in ASCII sendet, könnte zwischenzeitlich nach Baudot wechseln, um zu überprüfen wie voll der Puffer ist.

Communication Specification for i-Telex

When in a “texting baudot” state a “reject” or “end” end package returns the i-telex service to the disconnected state and terminates the tcp session.

In the state “disconnected baudot” the connection waits for a connect from the very same IP address as before. All other connects to the connected telex will be rejected. If a connection from the same IP arises the previous state is restored. If there is no subsequent connection within 30s the state changes to disconnected.

When in “texting ascii” state a termination of the tcp session changes the state to “disconnected”.

2.1 Basic characteristics

The communication is based on the TCP protocol. The port can be chosen individually; the existing systems use port 134.

The transmitted data is symmetric, i.e. the caller and the called stations use the same data protocol.

The transmitted data is organized in “packets”. Each packet consists of:

- 1 byte command
- 1 byte data length
- n byte data

Exception: In texting ASCII state, the byte codes 0A, 0D, 20-7E and A0 to FF (hex) are treated as plain ascii resp. ISO 8859-1 codes if they are not part of the data block as specified above.

A TCP packet may consist of one or more data packets and / or ASCII data.

The calling station always initiates the data transfer, so the calling station always sends the first data. The called station is ‘silent’ up to the first reception. The called station determines the protocol type according the first received data: If the first byte is a 0A, 0D or in range of 20-7E or A0 to FF (hex), it’s ASCII mode, of not it’s Baudot mode.

Because the protocol is designed to allow also “maintenance communication” using the same port, the called station shall start the connected printer not before receiving an appropriate data packet (see below).

If a station receives a packet with unknown command code, the packet should be discarded. The length byte determines where the next packet starts.

If the data length of a received packet is higher than allowed the extra data must be discarded.

In “texting baudot” state at least every 4 seconds a packet should be sent (either a “baudot data”, “heartbeat” or an “acknowledge”, see below).

In baudot mode, any station may close the existing connection, then the calling station will try to reconnect within 30 seconds without any influence to the communication handshake. This feature allows in case of an unintended cutoff a reconnection.

Kommentiert [Son5]: Ergänzen: SI, SO für Ziffern/Buchstaben und sonst noch mehr???

Except certain packets (mentioned then) there is no defined order or handshake procedure. So both stations may send any packet at any time, and should accept any packet at any time but should not rely on the order of packets as presented in the examples.

2.2 Packet types

The following table gives an overview over all (so far) defined packet types:

Command (hex)	Use	Version	Data	Allowed data length (decimal)
00	Heartbeat	1	None	0
01	Direct dial	1	Local number	1
02	Baudot data	1	Baudot codes	1 to 50
03	End	1	None	0
04	Reject	1	Cause of rejection	0 to 20
06	Acknowledge	1	Number of received and processed baudot codes or ascii characters	1
07	Version	1	Version number	1 to 20
08	Self test	1	random payload	2 to 254
09	Remote config	1	PIN + sub command + config data	3 to 254

Table 1: Packet types and codes

2.2.1 Heartbeat packet

This packet type is used for periodic transmission if nothing else is to transmit. The receiver awaits periodic data to check if the connection is stable or not. Instead of this packet also the acknowledge packet is preferred (if applicable). There is no reponse packet to be sent for a heartbeat packet. The heartbeat packet may be used while in “connected” or in -“texting baudot” state.

- Example: 00 00
- Meaning: nothing to do

2.2.2 Direct dial packet

This packet is used to address a specific teleprinter, if the called station has more than one connected printer. The (one and only) data byte is the internal dial number of the “extension”. If a non-existing extension number is transmitted, the called station SHOULD use the main machine. Another option in case of an invalid extension is to send a “reject packet” and subsequently drop the TCP connection.

This packet shall be sent before the first “baudot data packet” or “acknowledge packet”.

- Example: 01 01 22 (hex)
- Meaning: try to connect the local printer with extension number 34 (decimal).

Because the i-Telex-System supports single digit extension numbers as well as double digit extension numbers the following coding rules apply:

Extension number in i-Telex-System	Code value (decimal)	Code value (hex)
00	100	64
01	1	01
02	2	02
...	...	...
99	99	63
0	110	6E
1	101	65
2	102	66
...	...	...
9	109	6D
none	0	00
invalid	> 110	> 6E

Table 2: Coding of extension numbers

The called station shall start the addressed printer after reception of this packet. The called station shall send either an baudot data packet (chapter 2.2.3) or an acknowledge Packet (chapter 2.2.6) when (and not before) the printer is successfully started.

If there is any failure, the called station shall respond with a reject packet (chapter 2.2.5).

The calling station shall send the direct dial packet before the first baudot data packet (chapter 2.2.3) and before the first acknowledge packet (chapter 2.2.6).

2.2.3 Baudot data packet

With a “baudot data” packet the “real” data is transferred. The baudot data (5 data bits; LSB is the last bit transmitted; MSB is the first transmitted bit on the telex line) are padded with zero bits to 8 bits. **The length must not be more than 50 bytes.**

- Example: 02 07 1F 01 10 14 01 02 08 (hex)
- Meaning: print <Ltrs> T E S T <CR> <LF>

Regarding the interpretation of the serial data it is important to know, that the first data bit sent by the teleprinter is interpreted as the most significant bit (bit 4) and the last bit sent as last significant bit (bit 0). See the letter “T” sent as code 01.

The data is buffered at the receiving i-telex station. To prevent a buffer overflow, the sender SHOULD NOT cause a condition in which there are more than 254 bytes unacknowledged (See the following chapter 2.2.6).

If the called station didn’t receive a direct dial packet before the first baudot data packet it shall activate the default printer device.

2.2.4 Communication end packet

This packet indicates a normal termination of a connection (“hang-up”). The receiving station should switch off the printer and should await a close of the TCP connection. This packet can be sent by both stations (caller or called), whichever station terminates the connection at first.

- Example: 03 00 (hex)
- Meaning: connection will be closed

It's not necessary to answer the communication end packet, the station which receives this packet may immediately close the TCP connection.

2.2.5 Communication reject packet

This packet indicated an abnormal termination, for example a busy sign or an error message. The receiving station should close the connection. The data within this packet consists of ascii codes.

- Example: 04 03 6F 63 63 (hex)
- Meaning: the called station is momentary in use.

The following reject codes are defined, but other codes may be used.

Code (ASCII)	Code (hex)	Meaning
occ	6F 63 63	Occupied by another call
abs	61 62 73	i-telex station temporarily disabled
na	6E 61	Not allowed (called extension is not allowed for external connections)
nc	6E 63	Not connected (called extension is temporarily switched off)
der	64 65 72	Derailed (the i-telex station works not properly)

Table 3: Reject codes

It's not necessary to answer the communication reject packet, the station which receives this packet may immediately close the TCP connection.

2.2.6 Acknowledge packet

Using this packet, the station indicates the number of baudot characters (as received by the baudot data packet, see 2.2.3 or by ascii characters) which were received since establishing the connection. This packet has the purpose that a (sending) station may determine if all sended data was already processed by the receiving station or (if not) how many characters are currently buffered at the receiving station. So, this data packet allows to optimize the sending speed. The counting of characters resp. codes is independent of the communication direction.

- Example: 06 01 51 (hex)
- Meaning: currently 81 (decimal) baudot codes / characters have been processed by the printer since establishing the connection.

If the number of processed codes exceeds 255, the counting starts at 0 again.

This acknowledge packet shall not be send in case the local printer device is not running. Especially the called station shall not send any acknowledge packet before reception of an direct dial packet or baudot data packet.

On the other hand side, the called station must send an acknowledge packet after starting the printer device in case the called station does not send any baudot data after startup.

As an example, the standard i-Telex station processes this telegram in the following way:

Communication Specification for i-Telex

- For the sending direction, the i-Telex at first buffers all typed characters in a buffer. The count of buffered codes is N_buffer.
- The time since the last typed character is measured (T_typingpause)
- The i-Telex counts the transmitted baudot codes (N_trans).
- The value of the (last) acknowledge packet received from the opposite station is stored locally (N_ack).

The baudot data packet will be send, if:

- (N_buffer > 0) and (T_typingpause >= 1 second)
- OR (N_buffer > 0) and (N_ack == N_trans)
- OR (N_buffer >= 16)

Note: Normally the called i-Telex station prints out the current date and time after starting the local printer. To indicate that there is some data to print the first acknowledge data packet sent by the called station doesn't start with data byte of 0. But as soon as this locally generated text is completely printed the called station sends the acknowledge data packet with value 0 (so 06 01 00).

2.2.7 Version packet

Using this packet, the connected stations can handle out which version of the i-telex protocol is supported. Currently, there is only version 1 defined, but in future there may be enhanced versions. If a station receives a version packet with a version number it can handle, the station should "answer" with a version packet including the same version number as received. If the station does not support that version, it must respond with a version packet including a version number it supports. Caution: It may occur that this "proposal" is again denied by the "original" station and another ("lower") version will be handled out.

The version number is the first byte in the data block. The data block may contain further information, but this info is currently not relevant regarding the protocol. It may be used in future to signal available protocol extensions. This packet should be the sent before any packets defined only for higher versions and before any "data packet".

- Example: 07 05 01 35 32 34 00 (hex)
- Meaning: protocol version 1, additional info "524" (ascii code).

2.2.8 Self test packet

This packet is used like a "ping" signal to determine if the connection to the world wide web is working, especially regarding the port-forwarding for "incoming calls".

- Example: 08 02 45 6B (hex)
- Meaning: ping signal with random number 6B45 (hex).

Note: The self test is performed by sending this data packet to itself (to the own station, so to check if a known public IP address is still routed to the i-telex-station performing this test).

So, if any implementation has no need to perform the check, there is no need to implement the functionality for sending or receiving the self test package.

2.2.9 Remote configuration packet

This packet is only used for i-telex internal functions. It can be discarded. But this is the reason behind the rule not to start the printer device before reception of direct dial packet oder baudot data packet.

2.3 Examples

The following example shows the data exchange in a standard connection.
A is the calling station; B is the called station.

A → B (hex)	B → A (hex)	Meaning
07 05 03 38 37 35 00		A calls B with protocol version 3 (and some additional data)
	07 01 02	B replies that this station only supports protocol version 2 or lower.
07 01 01		A replies that it wants to use the basic protocol (e. g. if it supports only versions 3 and 1)
	07 01 01	B confirms the use of protocol version 1
01 01 22		A selects the printer device extension number 34 (decimal) at the “B” side.
	06 01 00	B confirms the successful start of the local printer device Alternatively a baudot data packet with “greetings” may be sent
02 07 1F 01 10 14 01 02 08 06 01 00		A itself starts the local printer device A sends text “test” with CR and LF to B and also states that no baudot data was received yet
	06 01 02	B states that the local printer is running but only two characters had been printed yet.
06 01 00		A sends acknowledge of zero processed codes (and maybe waits for B to process the data)
	06 01 07	B states that 7 codes were processed.
00 00		Alternative for sending “acknowledge” is “heartbeat”.
	02 02 02 08	B sends itself <CR> <LF>.
06 01 02		A states that <CR> <LF> were sent to the printer.
	06 01 07	B repeats “7 codes processed”
06 01 02		obviously at the moment nobody wants to send anything
	03 00	B announces the close of the connection
(close)		A closes the TCP connection and stops the local printer device

Table 4: Complete example

After the announcement of a connection close the opposite station may disconnect the TCP connection immediately.

Communication Specification for i-Telex

The following example is simplified for a connection, where only the sender transmits data to be printed.

A → B (hex)	B → A (hex)	Meaning
07 01 01		A calls B with protocol version 1
	07 01 01	B confirms the use of protocol version 1
02 07 1F 01 10 14 01 02 08		A sends text "test" with CR and LF to B B shall start the local default printer device
	06 01 02	B states that the local printer is running but only two characters had been printed yet.
00 00		A sends "heartbeat" (should occur each 2 seconds or alternatively as a reply to each packet received from B)
	06 01 07	B states that 7 codes were processed.
00 00		A sends "heartbeat"
	06 01 07	B repeats "7 codes processed"
03 00		A announces the close of the connection
	(close)	B stops the local printer device and closes the TCP connection.

Table 5: Simplified example

Again: As mentioned above it is allowed that station A or station B sends more than one packet without waiting for any reply of the opposite.

3 Communication between i-telex station and subscriber server

This chapter describes the necessary data exchange between the i-telex stations and the i-telex “nameserver”. The communication is based on the TCP protocol. The port which is used is 11811.

Example are given with hex values.

3.1 Basic characteristics

Every Packet has the same outline:

- Packet type (1 byte)
- Length (1 byte)
- Data (length bytes)
- Multiple byte integer values are sent with the least significant byte first (little endian).
- Strings are terminated by a NUL character (C style)

Exception: To provide a simple way for third-party systems the query of a peer is also supported in ASCII-Format. See chapter 3.3.

3.2 Packet types

Command (hex)	Use	Version	Data	Allowed data length (decimal)
0x01	Client_update	1	<Call-number 4b> <PIN 2b> <Port 2b>	8
0x02	Address_confirm	1	<IPAdr 4b>	4
0x03	Peer_query	1	<Call-number 4b> <Version 1b>	5
0x04	Peer_not_found	1	- no data -	0
0x05	Peer_reply_v1	1	<Call-number 4b> <Name 40b> <Flags 2b> <Type 1b> <Addr 40b> <IPAdr 4b> <Port 2b> <Extension 1b> <DynPin 2b> <Date 4b>	100
0x06	Sync_FullQuery	1	<Version 1b> <ServerPIN 4b>	5
0x07	Sync_Login	1	<Version 1b> <ServerPIN 4b>	5
0x08	Acknowledge	1	- no data -	0
0x09	End_of_List	1	- no data -	0
0x0a	Peer_search	1	<Version 1b> <Search pattern 40b>	41
0xFF	Error	1	<Diagnostic message> <NUL 1b>	x

Table 2: Packet types and codes

3.2.1 Client_update

This packet is used by a client to (if necessary) update the own IP address stored at the subscriber list. (a very simplified version of a “Dynamic DNS Server”).

Communication Specification for i-Telex

The data is structured as following:

- 1 byte Packet type: 01
- 1 byte length: 08
- 4 byte call-number: The own number of the client as a 32 bit integer value
- 2 byte PIN number: A number chosen by the user to authenticate
- 2 byte port number: The port number opened for incoming calls (usually 134).
- Example: 01 08 16 AA 34 00 34 12 86 00 (hex)
- Meaning: update the (dynamic) IP for the peer with number 3451414 with authentication PIN 4660 and port 134.

3.2.2 Address_confirm

This packet is sent by the server as a reply to a Client_update packet.

The data is structured as following:

- 1 byte Packet type: 02
- 1 byte length: 04
- 4 byte IP address: The IP address of the sender of packet "Client_update" which is stored in the subscriber list.
- Example: 02 04 xx xx xx xx (hex)
- Meaning: The sender has IP address xx xx.

3.2.3 Peer_query

This packet is sent by the client to query a subscriber server for contact details of a subscriber number

The data is structured as following:

- 1 byte Packet type: 03
- 1 byte length: 05
- 4 byte call-number: The (unique) number of the peer as a 32 bit integer value
- 1 byte version number: The highest version ID supported by the client. But currently the server supports only version 1.
- Example: 03 05 16 AA 34 00 01 (hex)
- Meaning: get the details of the peer with the number 3451414 with reply format version 1.

When sending this packet to a subscriber server the subscriber server replies either with a peer_not_found or a peer_reply_v1.

Note: Older clients do not provide the 1 byte version number. In this case version 1 is valid and the length byte is 04.

3.2.4 Peer_not_found

This packet is a possible reply to peer_query or peer_search packets. It is sent if the requested peer cannot be found or the result set of a peer search is empty.

- 1 byte Packet type: 04
- 1 byte length: 0

- Example: 04 00 (hex)
- Meaning: unable to find requested peers

3.2.5 Peer_reply_v1

This packet is the standard reply to a peer_query or peer_search packet, if the version number is 1 or the server does not support higher versions.

The data is structured as following:

- 1 byte Packet type: 05
- 1 byte length: 64 (hex)
- 4 byte call-number: The (unique) number of the peer as a 32 bit integer value
- 40 byte name: The “long” identification of the peer (as shown in the subscriber list), coded in ASCII resp. ISO 8859-1.
- 2 byte flags: Special “attributes” of the peer, encoded as a bit-masks in a 16 bit integer value:
 - Bit 0 (mask 0x01): always 0¹
 - Bit 1 (mask 0x02): always 0²
- 1 byte type: connection type of the peer:
 - 0: peer was deleted from the list
 - 1: peer able to support the “texting baudot” protocol, accessible by a host name (known by official DNS servers)
 - 2: peer able to support the “texting baudot” protocol, accessible by a given IP address (IPv4)
 - 3: peer only supporting “ascii texting” (or a standard telnet client), accessible by a host name (known by official DNS servers)
 - 4: peer only supporting “ascii texting” (or a standard telnet client), accessible by a given IP address (IPv4)
 - 5: same as 2, but IP address may change frequently
 - 6: not a real peer, but an “official” email address.
- 40 byte host name:
 - if “type” is 1 or 3: host name of the peer, encoded in ASCII resp. ISO 8859-1.
 - if “type” is 6: email address, encoded in ASCII resp. ISO 8859-1.
 - Invalid for other “types”.
- 4 byte IP address:
 - if “type” is 2, 4 or 5: IP address of the peer, coded as a 32 bit integer value.
 - Invalid for other “types”.
- 2 byte port number:
 - If “type” is 1 to 5: port to be used for the connection (standard 134), coded as a 16 bit integer value.
 - Invalid for other “types”.
- 1 byte extension:
 - if “type” is 1, 2 or 5: desired extension ID (see chapter 2.2.2, especially the coding!) or 0 if this entry is assigned to the main machine of the subscriber.
 - Invalid for other “types”.
- 2 bytes PIN: always 0³

Kommentiert [Son6]: Reihenfolge noch festlegen!

¹ This bit is used only internally as a flag for „private“ entries which are not subject to be reported by a query.

² This bit is used only internally as a flag for „hidden“ entries which will be synchronized between the subscriber servers but not listed in a query.

³ This data is used for the synchronisation communication between the subscriber servers.

- 4 byte date: Timestamp of the last change of this entry, encoded as a 32 bit integer value (seconds since 1900-01-00 00:00:00)

3.2.6 Sync_FullQuery

This packet is used only for the synchronisation between the subscriber servers. This packet initiates a full query of all entries stored at the “opposite” server station

The data is structured as following:

- 1 byte Packet type: 06
- 1 byte length: 05
- 1 byte version number: The highest version ID supported by the server. But currently the server supports only version 1.
- 4 byte Server-PIN: This PIN is unique for all “approved” servers. Server will answer only in case of correct PIN.
- Example: 06 05 01 16 AA 34 00 (hex)
- Meaning: Start the full query of the server with the authorisation PIN 3451414 and reply with format version 1.

3.2.7 Sync_Login

This packet is used only for the synchronisation between the subscriber servers. This packet initiates the update of a newly changed or added entry.

The data is structured as following:

- 1 byte Packet type: 07
- 1 byte length: 05
- 1 byte version number: The highest version ID supported by the server. But currently the server supports only version 1.
- 4 byte Server-PIN: This PIN is unique for all “approved” servers. Server will answer only in case of correct PIN.
- Example: 07 05 01 16 AA 34 00 (hex)
- Meaning: “Login” to the neighbored server with the authorisation PIN 3451414.

3.2.8 Acknowledge

This packet has to be sent as a general “ok”, for example by the client after the reception of a Peer_reply_v1 packet to request the next entry. Only to be used if the request was initiated by a Peer_search packet.

- 1 byte Packet type: 08
- 1 byte length: 00
- Example: 08 00 (hex)
- Meaning: Last reply received and processed, request to send next peer data according to query.

3.2.9 End_of_List

This packet is sent by the server either after a Peer_search packet or Acknowledge packet to indicate that all enties matching the requested name pattern were already transmitted.

Kommentiert [FS7]: Prüfen

Communication Specification for i-Telex

- 1 byte Packet type: 09
- 1 byte length: 00
- Example: 09 00 (hex)
- Meaning: No further peer matches to the query (or no match at all).

3.2.10 Peer_serach

This packet is sent by the client to query a subscriber server for contact details matching a given “name”.

- 1 byte Packet type: 0a (hex)
- 1 byte length: 29 (hex)
- 1 byte version number: The highest version ID supported by the client. But currently the server supports only version 1.
- 40 byte name: A part of the identification of the peer which should match the name as shown in the subscriber list, coded in ASCII resp. ISO 8859-1.
- Example: 0A 29 01 53 31 30 30 00 (hex)
- Meaning: search for subscriber list entries containing “T100”.

3.2.11 Error message

This packet indicates any error. In most cases the connection will be terminated directly after sending the error message.

- 1 byte Packet type: ff (hex)
- 1 byte length: variable
- x byte message string: any diagnostic message encoded in ASCII resp. ISO 8859-1.
- 1 byte null terminating character: 00 (hex)

The length field will contain (as usual) the full length of the message telegram, so the length of the diagnostic message incremented by 3 (x+3).

3.2.12 Sequence Examples

Unlike the i-Telex protocol for “normal” communication the server related protocol is always a kind of “ping pong”. In the following examples “O” stands for the originating part of the connection and “A” for the answering part. “A” is always a server, “O” may be a client or a server.

Client “DynIP” update:

O: Client_update

A: Address_confirm

(O terminates the connection)

Client query for specific entry:

O: Peer_query

if (entry found)

A: Peer_reply_v1

else

Communication Specification for i-Telex

A: Peer_not_found
(O terminates the connection)

Client query according search pattern:

O: Peer_search
for each corresponding entry:
A: Peer_reply_v1
O: Acknowledge
next
A: End_of_List
(O terminates the connection)

Full query between servers:

O: Sync_FullQuery
for each stored entry
A: Peer_reply_v1
O: Acknowledge
end while
A: End_of_List
(O terminates the connection)

Update of some entries from server to server:

O: Sync_Sync_Login
A: Acknowledge
for each stored entry
O: Peer_reply_v1
A: Acknowledge
end while
O: End_of_List
(O waits 3 seconds and terminates the connection)

3.3 Peer query in ASCII format

To simplify queries of subscribers for third party systems the following way is implemented:

Send the letter q and the required number in ASCII format followed by CR and LF (0D 0A) to port 11811 and the server will answer also in ASCII the following lines (also separated by CR and LF):

Communication Specification for i-Telex

Example reply	Explanation
ok	Indication that subscriber number was found
781272	Repetition of the subscriber number
Fred, Braunschweig T100	Subscriber's name
1	Type on entry: ○ 1: peer able to support the "texting baudot" protocol, accessible by a host name (known by official DNS servers) ○ 2: peer able to support the "texting baudot" protocol, accessible by a given IP address (IPv4) ○ 3: peer only supporting "ascii texting" (or a standard telnet client), accessible by a host name (known by official DNS servers) ○ 4: peer only supporting "ascii texting" (or a standard telnet client), accessible by a given IP address (IPv4) ○ 5: same as 2, but IP address may change frequently ○ 6: not a real peer, but an "official" email address.
sonnibs.no-ip.org	Host name for types 1 and 3 IP address for types 2, 4 and 5 Email address for type 6
134	Port number
33	Extension number, 0 for no extension
+++	Marker for end of data

Table 2: Example for ASCII format query reply

The answer for a query to an unknown number will be:

```
fail
12345
unknown
+++
```

It's important that the full query string (in example q781272<CR><LF>) is sent in a single TCP frame.

4 Backup: Basics of the communication

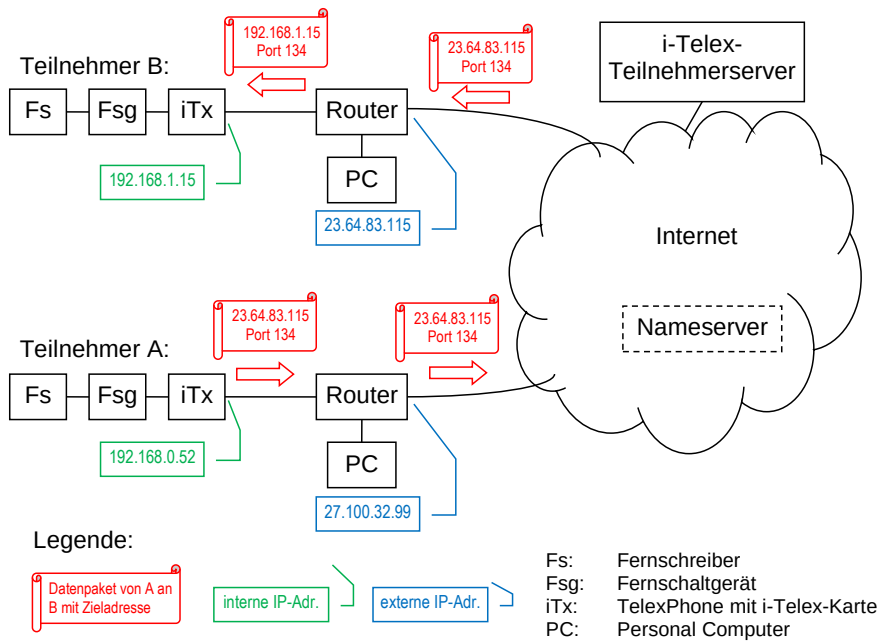


Abbildung 1: Example structure